

On-Chain Anti-Scalping Ticketing System — v5

Architecture & Implementation Plan

작성일: 2026-08-09 대상: Claude Code (구현 담당) 프로젝트 경로: `~/Documents/Solana Summer School/solana-capstone-project` 프로그램명: `ticketing-system` (package) / `ticketing_system` (crate) **Anchor** 버전: 1.0.1 (실제 resolve: anchor-lang/anchor-spl v1.1.2) **Demo Day**: 2026-08-17 (D-7 as of 작성일)

0. 프로젝트 핵심 논제

암표 방지법(재판매 가격 상한)은 존재하지만 기술적 강제 수단이 없다. 이 프로젝트는 Solana 온체인에서 재판매 트랜잭션 자체가 가격 상한을 넘으면 **실행 자체가 불가능**하도록 만들어, "불법"을 "구조적으로 불가능"으로 바꾼다.

핵심 기술 결정: Token-2022의 **Transfer Hook**이 아닌 **Default Account State (Freeze/Thaw)** 를 채택. Transfer Hook은 `amount: u64`로 토큰 수량만 볼 수 있고 결제 금액을 볼 수 없어 가격 강제가 구조적으로 불가능. Freeze/Thaw는 단일 원자적 instruction(`execute_resale`) 안에서 thaw → 가격검증 → USDC결제 → 티켓이동 → freeze를 모두 강제할 수 있음.

1. 이미 완료된 것 (건드리지 말 것)

- Anchor 스캐폴딩 완료 (`anchor init` 기반), 빌드/테스트(LiteSVM) 통과 확인됨
- `rust-toolchain.toml`: channel 1.89.0, components에 `rust-analyzer` 포함
- `Cargo.toml`: `anchor-lang`, `anchor-spl` (feature: `token_2022_extensions`) 의존성 추가됨
- GitHub 레포 연결 및 첫 커밋 완료
- 기존 파일: `programs/ticketing-system/src/{lib.rs, state.rs, constants.rs, error.rs, instructions.rs, instructions/initialize.rs}, tests/test_initialize.rs`

주의: `state.rs`, `constants.rs`, `error.rs`는 현재 비어있거나 `anchor init` 기본 템플릿 상태. 아래 v5 스펙으로 새로 작성.

2. 확정된 설계 결정 (더 이상 논의 불필요, 그대로 구현)

항목	결정
좌석 구조	지정석 지원 필수 — 등급(Tier)별 mint, 좌석(Seat)별 개별 계좌
좌석 표기	경기장마다 다른 자유 형식 문자열 지원 (예: "3-A", "동측 3구역 15열")
환불	필수 기능. Organizer 자율 정책(단일 마감일, 개인 사정 포함)으로 단순화
체크인 서명	스태프(Organizer 지갑 검직) 서명. 별도 VenueAuthority PDA는 만들지 않음
Bid queue	핵심 스코프로 확정, 구현함. 고정 크기 배열(<code>MAX_QUEUE_LEN</code>) 기반 FIFO로 최소 구현. Buyer는 <code>join_queue</code> 에서 USDC를 미리 예치(escrow)하고, seller는 <code>execute_resale</code> 단독 서명으로 큐 맨 앞과 자동 매칭

항목	결정
Designated transfer	이번 스코프에서 구현하지 않음 (시간 되면 추후 확장) — bid queue가 핵심 재판매 경로가 됨에 따라 우선순위 하락
Primary purchase	<code>buy_seat()</code> 단일 원자적 함수로 결제+소유권 확정을 한 트랜잭션에서 처리 (D-1 결정)
Seat PDA 생성 방식	전량 사전 생성 (Organizer가 이벤트 오픈 시 좌석 PDA를 미리 다 <code>init</code>). "새로고침하며 빈자리 찾기" UX 지원 위함
프론트엔드 지갑 연동	Demo Day 발표는 devnet에 미리 세팅한 테스트 키페어로 시나리오 재생 (실제 Phantom 연동은 언급만)

3. PDA 구조 (v5)

```

JurisdictionRegistry (지역당 1개, 기존 유지)
Event (이벤트당 1개)
├── SeatTier (등급당 1개, 예: VIP/R/S)
│   ├── BidQueue (등급당 1개, FIFO 대기줄 + USDC vault)
│   └── Seat (좌석당 1개, 예: "3-A")
│       └── TokenAccount (Token-2022 표준, 실제 소유권)

```

논리적 계층 표현일 뿐, 실제로는 4~5종류의 독립 계좌가 Pubkey로 서로 참조하는 flat 구조.

3-1. JurisdictionRegistry (기존 유지, 참고용)

```

#[account]
pub struct JurisdictionRegistry {
    pub authority: Pubkey,
    pub jurisdiction_code: String,
    pub legal_cap_bps: u16,
    pub bump: u8,
}

```

3-2. Event

```

#[account]
pub struct Event {
    pub organizer: Pubkey,
    pub event_id: u64,
    pub jurisdiction_registry: Pubkey,
    pub refund_deadline: i64, // unix timestamp, 이 시점까지만 환불 가능
    pub refund_bps: u16, // 10000 = 전액 환불
    pub bump: u8,
}

```

```
impl Event {
    pub const SPACE: usize = 8 + 32 + 8 + 32 + 8 + 2 + 1;
}
```

- seeds: [b"event", organizer.key(), &event_id.to_le_bytes()]

3-3. SeatTier

```
#[account]
pub struct SeatTier {
    pub event: Pubkey,
    pub tier_name: String,          // "VIP", "R", "S" 등, max 16 bytes
    pub ticket_mint: Pubkey,       // Token-2022, decimals=0,
    DefaultAccountState=Frozen
    pub payment_mint: Pubkey,      // USDC mint
    pub face_value: u64,          // USDC base units
    pub resale_policy: ResalePolicy, // policy_snapshot 결과, 이 시점에 확정
    pub total_seats: u32,
    pub bump: u8,
}

impl SeatTier {
    pub const SPACE: usize = 8 + 32 + (4 + 16) + 32 + 32 + 8 +
    ResalePolicy::SPACE + 4 + 1;
}
```

- seeds: [b"seat_tier", event.key(), tier_name.as_bytes()]
- **중요:** policy_snapshot (= min(organizer_policy, jurisdiction_legal_cap) via more_restrictive())이 이 instruction(**create_seat_tier**) 실행 시점에 계산되어 저장됨. v4에서는 create_event 시점이었으나 v5에서 이동.

3-4. ResalePolicy (기존 유지)

```
#[derive(AnchorSerialize, AnchorDeserialize, Clone, Copy, PartialEq, Eq,
Debug)]
pub enum ResalePolicy {
    Capped { max_bps: u16 },
    Unrestricted,
    NonTransferable,
}

impl ResalePolicy {
    pub const SPACE: usize = 1 + 2;

    pub fn more_restrictive(a: ResalePolicy, b: ResalePolicy) ->
    ResalePolicy {
        use ResalePolicy::*;
        match (a, b) {
            (NonTransferable, _) | (_, NonTransferable) => NonTransferable,
```

```

        (Capped { max_bps: a_bps }, Capped { max_bps: b_bps }) => {
            Capped { max_bps: a_bps.min(b_bps) }
        }
        (Capped { max_bps }, Unrestricted) | (Unrestricted, Capped {
max_bps }) => {
            Capped { max_bps }
        }
        (Unrestricted, Unrestricted) => Unrestricted,
    }
}
}

```

- 총순서(total order): `NonTransferable > Capped{max_bps 작을수록 엄격} > Unrestricted`
- 명시적 `match` 패턴으로 구현 (컴파일타임에 미처리 케이스 검출 가능하게)

3-5. Seat

```

#[derive(AnchorSerialize, AnchorDeserialize, Clone, Copy, PartialEq, Eq,
Debug)]
pub enum SeatStatus {
    Available,
    Sold,
    Refunded,
    CheckedIn,
}

#[account]
pub struct Seat {
    pub seat_tier: Pubkey,
    pub seat_code: String,           // PDA seed용, max 16 bytes (예: "3A15")
    pub display_name: String,       // 자유 형식, max 64 bytes (예: "동측 3구역 15
열")
    pub token_account: Pubkey,      // 연결된 TokenAccount 주소 (참조, 소유 아님)
    pub owner: Pubkey,              // 기록용 (실제 권한 근거는 TokenAccount.owner)
    pub status: SeatStatus,
    pub bump: u8,
}

impl Seat {
    pub const SPACE: usize = 8 + 32 + (4+16) + (4+64) + 32 + 32 + 1 + 1;
}

```

- seeds: `[b"seat", event.key(), seat_tier.key(), seat_code.as_bytes()]`
- 주의: PDA seed는 계좌당 최대 32바이트 제한. `seat_code`는 짧게, 사람이 읽는 이름은 `display_name`에 자유롭게.

3-6. BidQueue (신규 — FIFO 대기줄, 핵심 스코프)

```
#[derive(AnchorSerialize, AnchorDeserialize, Clone, Copy, PartialEq, Eq, Default)]
pub struct Bid {
    pub buyer: Pubkey,
    pub amount: u64,          // 예치된 USDC (제시 가격)
    pub joined_at: i64,
}

#[account]
pub struct BidQueue {
    pub seat_tier: Pubkey,
    pub vault: Pubkey,        // 예치 USDC 보관 토큰계좌, authority = BidQueue PDA
    pub bids: [Bid; MAX_QUEUE_LEN], // 고정 크기 배열, index 0 = 맨 앞(FIFO)
    pub count: u8,
    pub bump: u8,
}

impl BidQueue {
    // 8(disc) + 32(seat_tier) + 32(vault) + MAX_QUEUE_LEN*(32+8+8) + 1(count) + 1(bump)
    pub const SPACE: usize = 8 + 32 + 32 + (MAX_QUEUE_LEN * (32 + 8 + 8)) + 1 + 1;
}
```

- seeds: [b"bid_queue", seat_tier.key()]
- MAX_QUEUE_LEN은 constants.rs에 정의 (데모 목적으로 작게, 예: 20)
- 고정 배열을 쓴 이유: Solana 계좌는 동적 크기 조정(**realloc**)이 가능하지만 시간 제약상 복잡도를 낮추려고 고정 크기로 단순화. 실 서비스라면 가변 크기나 별도 PDA-per-bid 구조로 확장 필요 (발표에서 "다음 단계"로 언급 가능)
- 맨 앞 체결 후 **bids** 배열을 한 칸씩 당기는(shift) 방식으로 dequeue 구현

3-7. TokenAccount (Token-2022 표준, 새로 정의하지 않음)

- **mint**: 해당 **SeatTier.ticket_mint**
- **owner**: 구매자 지갑 — **실제 소유의 유일한 근거**
- **amount**: 항상 0 또는 1 (decimals=0)
- **state**: **Frozen** (기본값) / **Initialized** (execute_resale 실행 중 잠시만)

4. Constants / Errors

```
// constants.rs
pub const EVENT_SEED: &[u8] = b"event";
pub const SEAT_TIER_SEED: &[u8] = b"seat_tier";
pub const SEAT_SEED: &[u8] = b"seat";
pub const TICKET_MINT_SEED: &[u8] = b"ticket_mint";

pub const MAX_TIER_NAME_LEN: usize = 16;
pub const MAX_SEAT_CODE_LEN: usize = 16;
```

```
pub const MAX_DISPLAY_NAME_LEN: usize = 64;
pub const MAX_QUEUE_LEN: usize = 20; // BidQueue 고정 슬롯 수 (데모용, 실서비스
는 확장 필요)
```

```
// error.rs
#[error_code]
pub enum TicketError {
    #[msg("max_bps must be between 0 and 10000")]
    InvalidMaxBps,
    #[msg("seat is not available for purchase")]
    SeatNotAvailable,
    #[msg("seat code exceeds max length")]
    SeatCodeTooLong,
    #[msg("display name exceeds max length")]
    DisplayNameTooLong,
    #[msg("tier name exceeds max length")]
    TierNameTooLong,
    #[msg("resale price exceeds policy cap")]
    ResalePriceExceedsCap,
    #[msg("refund deadline has passed")]
    RefundDeadlinePassed,
    #[msg("ticket already checked in or invalid state for this operation")]
    InvalidSeatStatus,
    #[msg("caller is not authorized organizer")]
    Unauthorized,
    #[msg("bid queue is full")]
    QueueFull,
    #[msg("bid queue is empty, nothing to resell")]
    QueueEmpty,
    #[msg("bid amount exceeds resale policy cap")]
    BidExceedsCap,
}
```

5. Instruction 목록 (구현 순서대로)

5-1. create_event

- Organizer가 서명, Event PDA 생성
- 파라미터: event_id: u64, jurisdiction_registry: Pubkey, refund_deadline: i64, refund_bps: u16

5-2. create_seat_tier

- Organizer가 서명, SeatTier PDA 생성 + Token-2022 mint 동시 생성 (decimals=0, mint::authority=seat_tier, mint::freeze_authority=seat_tier, extensions::default_account_state::state = AccountState::Frozen)
- 파라미터: tier_name: String, face_value: u64, organizer_resale_policy: ResalePolicy, total_seats: u32, payment_mint: Pubkey

- 로직: `JurisdictionRegistry`를 읽어서 `legal_cap_bps` 확인 → `ResalePolicy::more_restrictive(organizer_resale_policy, Capped{legal_cap_bps})` 계산 → `resale_policy` 필드에 저장 (`policy_snapshot`)
- `max_bps <= 10000` 검증 (`InvalidMaxBps`)

5-3. `create_seat`

- Organizer가 서명, `Seat` PDA 생성 (사전 생성 방식 — 이벤트 오픈 시 좌석 수만큼 반복 호출)
- 파라미터: `seat_code: String, display_name: String`
- `status = Available`로 초기화, `token_account/owner`는 아직 비어있음 (또는 placeholder)
- 길이 검증: `seat_code.len() <= MAX_SEAT_CODE_LEN, display_name.len() <= MAX_DISPLAY_NAME_LEN`

5-4. `buy_seat` (핵심 — Primary purchase, 원자적)

- Customer가 서명
- 한 트랜잭션 안에서 원자적으로 처리:
 1. `Seat.status == Available` 확인 (`SeatNotAvailable`)
 2. Customer → `SeatTier`(또는 Organizer)로 `face_value`만큼 USDC transfer
 3. Customer의 `TokenAccount`에 `mint_to(amount=1)` (Frozen 상태로 생성됨 — Default Account State 덕분에 자동)
 4. `Seat.token_account = 새로 만든 TokenAccount 주소, Seat.owner = customer, Seat.status = Sold`로 업데이트

5-5. `join_queue` (신규 — Bid queue 진입)

- Buyer가 서명
- 파라미터: `amount: u64` (제시 가격, USDC)
- 로직:
 1. `amount`가 `SeatTier.resale_policy` 상한 이내인지 검증 (`BidExceedsCap`) — 애초에 상한 넘는 입찰 자체를 막음
 2. `BidQueue.count < MAX_QUEUE_LEN` 확인 (`QueueFull`)
 3. Buyer → `BidQueue.vault`로 `amount`만큼 USDC transfer (escrow)
 4. `bids[count] = Bid { buyer, amount, joined_at: now }, count += 1`

5-6. `leave_queue` (선택 구현 — 시간 되면)

- Buyer가 서명, 자기 자리를 큐에서 제거하고 예치금 환불
- 시간 부족하면 스코프에서 제외 가능 (핵심 논제 증명에는 불필요)

5-7. `execute_resale` (핵심 — 논제 증명 지점, 큐 기반으로 재설계)

- **Seller(현재 좌석 소유자)만 서명** — buyer는 이미 `join_queue`에서 서명/예치를 마쳤으므로 이 시점엔 서명 불필요
- 한 트랜잭션 안에서 원자적으로:
 1. `BidQueue.count > 0` 확인 (`QueueEmpty`)
 2. `front = bids[0]` (맨 먼저 줄 선 사람) — **seller는 이 front를 임의로 바꿀 수 없음 (seller fills front only)**
 3. `thaw` (seller의 `TokenAccount`)

4. front.amount가 여전히 `SeatTier.resale_policy` 상한 이내인지 재검증 (`ResalePriceExceedsCap`) — 등록 이후 정책이 바뀌었을 가능성 대비
 5. `BidQueue.vault` → Seller로 front.amount만큼 USDC transfer
 6. Ticket transfer (seller TokenAccount → front.buyer의 TokenAccount, amount=1)
 7. `freeze` (seller TokenAccount and buyer TokenAccount 둘 다 — "re-freeze both accounts")
 8. `Seat.owner = front.buyer`로 업데이트
 9. 큐에서 맨 앞 제거: bids를 한 칸씩 당기고(shift) `count -= 1`
- 이 instruction 하나가 프로젝트 핵심 논제를 증명함: Transfer Hook으로는 불가능한 "결제 금액을 보고 트랜잭션을 거부"가 여기서 실현되고, 동시에 대기줄 강제(`seller fills front only`)로 판매자-구매자 간 사적 협상 채널 자체를 차단함

5-8. refund_ticket

- Organizer가 서명 (또는 Customer가 요청, Organizer 승인 — 단순화하려면 Organizer 단독 서명으로)
- 조건: `now <= Event.refund_deadline` (`RefundDeadlinePassed`), `Seat.status == Sold` (`InvalidSeatStatus`)
- 로직: Organizer → Customer로 `face_value * refund_bps / 10000`만큼 USDC 지급, `Seat.status = Refunded`, TokenAccount의 티켓 무효화 (burn 또는 상태 플래그만 변경 — burn 권장: 재사용 방지 명확함)

5-9. check_in

- Venue authority(=Organizer 지갑 겸직)가 서명
- 조건: `Seat.token_account`가 실제로 `amount==1`을 가진 계좌와 일치, `Seat.status == Sold`
- 로직: `Seat.status = Checkedin`으로 변경. 토큰 자체는 건드리지 않음 (소각 안 함 — 나중에 참석 증명 (POAP류)으로 활용 가능하도록 유지). Freeze/Thaw 불필요 (소유권 이동이 없으므로)

6. 테스트 (LiteSVM 기반, Rust)

각 instruction마다 최소 1개의 happy-path 테스트 + 핵심 실패 케이스 테스트:

- `create_event`, `create_seat_tier`, `create_seat`: 정상 생성 확인
- `create_seat_tier`: `more_restrictive()` 로직 검증 — organizer policy가 legal cap보다 느슨할 때 legal cap이 적용되는지, 반대의 경우도
- `buy_seat`: 정상 구매 시 USDC 이동 + TokenAccount 생성 + Frozen 상태 확인
- `join_queue`: 핵심 테스트 — (a) 정책 상한 이내 입찰이면 성공 + USDC escrow 확인, (b) 상한 초과 입찰이면 실패(`BidExceedsCap`), (c) 큐가 가득 찼을 때 실패(`QueueFull`)
- `execute_resale`: 핵심 테스트 — (a) 큐에 대기자가 있을 때 맨 앞과 정상 매칭, (b) 큐가 비었을 때 실패 (`QueueEmpty`), (c) 재판매 후 양쪽 TokenAccount가 다시 Frozen인지 확인, (d) 체결 후 큐에서 맨 앞이 제거되고 다음 사람이 새 front가 되는지 확인 (FIFO 순서 검증)
- `refund_ticket`: 마감일 이내/이후 각각 테스트
- `check_in`: 정상 체크인 + 중복 체크인 방지 확인

7. 이번 스코프에서 하지 않는 것 (명시적 제외)

- Designated transfer / TransferOffer PDA — 코드 미구현 (bid queue가 핵심 재판매 경로가 되면서 우선순위 하락)

- Bid queue의 가변 크기 확장 (**realloc**) — 고정 배열(**MAX_QUEUE_LEN=20**)로 데모 규모만 지원. 실 서비스 확장은 "다음 단계"로 언급
- VenueAuthority PDA (스태프별 개별 권한) — Organizer 지갑으로 검직
- Jurisdiction Registry와 연동된 환불 정책 — Organizer 자율 정책으로 대체
- 실제 지갑(Phantom) 연동 — devnet 테스트 키페어로 시나리오 재생

8. 프론트엔드 (별도 트랙, 온체인과 병렬 진행)

- 스택: React + **@solana/wallet-adapter-react** + Anchor IDL 자동생성 TS client
- 최소 데모 시나리오: Event/SeatTier/Seat 생성(관리자 화면) → 좌석 목록 보기 → 좌석 구매 → 큐 참여 (join_queue) → 재판매 시도(상한 초과 시 거부되는 것을 화면에 명확히 표시) → (시간 되면) 환불/체크인
- devnet 배포 + 테스트 키페어 여러 개 미리 준비 (organizer 1, customer 2~3개)

8-1. 온체인-프론트엔드 병렬 진행 타이밍 (중요)

완전히 순차적으로 진행하지 말 것. 프론트엔드는 **anchor build** 시 자동 생성되는

IDL(**target/idl/ticketing_system.json**)에 의존하므로, 온체인 instruction이 하나씩 컴파일 가능해질 때마다 프론트엔드도 그 시점 IDL로 이어서 작업한다:

```
create_event, create_seat_tier, create_seat, buy_seat 빌드 성공
→ 이 시점 IDL로 프론트엔드 스캐폴딩 즉시 시작
(지갑 연동, Event/Seat 생성 관리자 화면, 좌석 목록/구매 화면)

join_queue, execute_resale 빌드 성공 (핵심 논제 증명 지점)
→ IDL 갱신 → 프론트엔드에 큐 참여 + 재판매 시연 화면 연결
(상한 초과 시 트랜잭션이 거부되는 것을 시각적으로 명확히 보여주는 것이 데모의 핵심 장면)

refund_ticket, check_in 빌드 성공
→ IDL 갱신 → 프론트엔드에 순차 연결 (시간 되는 만큼)
```

이렇게 진행하면 온체인 작업이 다 끝나기 전에 프론트엔드가 이미 상당 부분 진행되어 있어, Day 6-7에 리허설/버그픽스에 집중할 시간을 확보할 수 있다. 온체인을 100% 끝낸 뒤에 프론트엔드를 시작하는 방식은 D-7 안에 완료 리스크가 크므로 지양할 것.

9. 우선순위 (D-7 기준, 시간 부족 시 자르는 순서)

1. 반드시: **create_event, create_seat_tier, create_seat, buy_seat, join_queue, execute_resale** — 핵심 논제(가격 강제) + 핵심 차별점(대기줄 강제) 증명에 필수
2. 가능하면: **refund_ticket, check_in, leave_queue**, 프론트엔드 데모 연결
3. 시간 남으면: 계단식 환불, 실제 지갑 연동, BidQueue 가변 크기 확장, Designated transfer

참고: bid queue가 핵심 스코프로 확정되면서 구현 항목이 늘어났으므로(**join_queue, execute_resale** 재설계), Day 3(execute_resale) 작업 범위가 예상보다 커질 수 있음. 진행하면서 하루 이틀 소요 시 **leave_queue**부터 먼저 잘라낼 것.